

Automated Pitch Boundary & Crop Engine

Software / Backend Developer Challenge

What it is: a pipeline that reads match video, finds the pitch boundary, and reports the run to the platform

Language: Python 3.11+ (container uses 3.12)

Tests: 176 passing

Runs in: Docker, reporting over the network

Pitch Boundary & Camera Crop Engine

This reads a match video, finds the pitch boundary in each frame, works out a result, and tells the platform how the run went.

It replaces the old research prototype. That prototype is still in `Legacy/`, untouched, so you can compare. See `DECISIONS.md` for why things are the way they are.

What you need

- Python 3.11 or newer (the container uses 3.12)
- Docker if you want to run it the way it would run in production
- A video. There is a script that makes a fake one for testing.

Running it locally

```
python -m venv .venv
.venv/Scripts/activate          # on Windows. On Mac or Linux use: source .venv/bin/activate
pip install -r requirements.txt

# Make a fake video first. Videos aren't committed to git.
python -c "from tools.synthetic_generator import generate_synthetic_video; generate_synthetic_video()"

# Run it. --no-reporting means don't try to talk to the platform.
python -m trackbox_pitch.cli --no-reporting --log-format console
```

Logs are JSON by default, because in production nobody is watching the screen. If you are a human reading them, add `--log-format console`.

Running it in Docker

```
python -c "from tools.synthetic_generator import generate_synthetic_video; generate_synthetic_video()"
docker compose up --build
```

This starts the reporting service first and waits until it is actually ready, then runs the pipeline. The pipeline talks to it over the network. No shared files, no database.

To see what the platform received:

```
curl -s http://localhost:5000/api/v1/jobs/events | jq
```

The video is mounted into the container instead of being copied into the image, so generated files never end up baked into an image.

Settings

`config/default.yaml` holds everything. Every field has to be there. There are no hidden defaults.

If something is missing, misspelled, or out of range, the program stops immediately and tells you which key was wrong. It does not run halfway and then fall over, and it does not quietly use a value you did not choose.

You can also override settings with environment variables. They go through the same checks as the file, so you cannot sneak in a bad value that way either.

Variable	Changes
<code>TRACKBOX_VIDEO_PATH</code>	<code>video.path</code>
<code>TRACKBOX_TARGET_FPS</code>	<code>video.target_fps</code>
<code>TRACKBOX_API_URL</code>	<code>reporting.api_url</code>
<code>TRACKBOX_LOG_LEVEL</code>	<code>logging.level</code>
<code>TRACKBOX_LOG_FORMAT</code>	<code>logging.format</code>

The file wins over nothing, then environment variables, then command line flags.

```
python -m trackbox_pitch.cli --help
```

Exit codes

Code	Meaning
0	Finished fine
1	Something unexpected broke. A bug.
2	The config is wrong
3	The video is missing, unreadable, or empty
4	The video is cut short or a frame would not decode
5	Ran fine, but found no pitch boundary anywhere

They are all different on purpose. A scheduler should be able to tell "bad config" from "bad video" from "ran fine but saw nothing" without reading any log text.

What it does

- Keeps the finding of boundaries separate from the running of things. The pipeline asks for a `FieldDetector` and does not know which one it gets. Detectors are picked by name from the config, so a typo in a detector name fails at startup.
- Only looks at frames it needs. A 1800 frame video gets looked at 300 times, not 1800. Frames it skips are never decoded, just stepped over.
- Does not keep anything that grows with the video length. Polygons are kept in a small ring buffer.
- Splits failures into "stop now" and "keep going", and counts every rejected frame with the reason. Nothing gets dropped without being counted.
- Sends reports built from proper models, and treats an unreachable platform as a degradation, not a failure. But it never lets that hide a real failure.

What it does not do

- No crop layout recommendation. The old prototype had a config block for it but never read it. I left that block out instead of copying decoration.
- The detector is not tuned. `green_threshold` is a straight port, known limitations and all. See `DECISIONS.md`.
- If the video resolution changes halfway through, it stops. Mixing up two coordinate systems would quietly ruin every number, so stopping is better.

Tests

```
python -m pytest -q
python -m ruff check src/ tests/
```

Where things live

src/trackbox_pitch/	
config.py	settings, and the checks that reject bad ones
errors.py	what can go wrong, and what exit code it means
models.py	the shapes that get passed around and sent to the platform
detectors/	the one part that swaps out: finding the boundary
video.py	deciding which frames to look at
aggregation.py	adding up the results
logging_setup.py	one JSON line per event
reporting.py	talking to the platform
pipeline.py	runs the whole job
cli.py	the command line entry point
tools/	makes the fake video. Not part of the shipped code.
legacy/	the old prototype, untouched
config/	default.yaml
tests/	the tests
docs/	design notes
mock_api/	the reporting service we were given. Not modified.

Decisions

Where I stopped, what I assumed, and what I would ask before this went anywhere near production.

The numbers below were all measured on this machine against the 1800 frame test video. None of them are estimates.

1. Assumptions and open questions

Things I assumed

- **The video is a file on disk.** The brief says the generator makes the input and my job starts from what it produces, so the pipeline takes a path. In production it is probably a stream or an object store key. Only `video.py` would need to change, because everything else just gets frames.
- **The frame rate and frame count from the file are roughly right.** I allow them to be off by 2% (explained in section 2).
- **The resolution stays the same for the whole video.** If it changes, I stop the run. Every measurement is in pixels. If half the video is at one size and half at another, all the numbers become meaningless.
- **The spatial metric is the one the prototype meant.** That is the area where the boundary overlaps the frame. I kept it, but I compute it once now instead of rebuilding the same rectangle on every frame.
- **confidence is a rough stand-in, not a real score.** A colour threshold cannot give you a real confidence. It reports how much of the frame matched the colour, and I named and documented it that way instead of pretending it is something better.

Things I would ask the product and ML teams

1. **Is it OK to look at 5 frames a second instead of 30?** This is the single biggest performance decision in here, and it throws away five out of every six frames. That is fine when the answer is a summary. It would be wrong if anything downstream needs every single frame. I would want an answer to this before anything else.
2. **What should "found nothing" mean to the scheduler?** I made it exit 5 with a `no_valid_detections` status. A whole game shot from a fixed camera with no pitch in view is a real outcome, not a crash. But if the platform treats it as missing data that should be retried, that is a different piece of work.
3. **If the video is cut short, should we stop or keep what we have?** I stop and exit 4, with the partial results attached. The other option is to finish processing what is there and mark the run as degraded. That is a product call about whether half a result is better than none.
4. **Who reconnects if the stream drops?** I treat a broken frame as fatal. If that is usually a blip in production, the scheduler should retry and my exit code tells it so. If the pipeline is supposed to reconnect itself, that is new work.
5. **Does anything actually use the polygon?** If consumers only want the summary numbers, the ring buffer and the consensus polygon can go and the code gets simpler. I kept them because the prototype

produced polygons and its README says they feed a crop step.

6. **Is crop layout in scope?** The prototype has a `crop_search` config block that nothing reads, and its README says crop layout is "not yet implemented". I did not build it. If it is needed, it should sit behind the same kind of seam as the detector.
7. **Multi-camera.** The prototype README talks about multi-camera video. I built one video per run and assumed the scheduler fans out. If one run is meant to handle several cameras at once, both the frame reader and the aggregator change.
8. **How wrong is acceptable?** The detector is broken in a specific way on this feed (section 2), so I cannot give you a real false positive rate. I would want a target before tuning anything.

2. Where I fail fast, and where I allow a fallback

Where it stops

What happened	What it does	Exit code
Config missing, unreadable, unknown key, bad value	Stops before opening anything	2
Video missing, unopenable, or has no frames	Stops	3
Video cut short, frame won't decode, resolution changes	Stops	4
Anything unexpected	Stops, with the traceback	1

Config is checked before the video is opened, so a bad config costs you nothing.

Every model rejects unknown keys. That means `min_are` instead of `min_area` is an error, and the message tells you both that `min_are` is not allowed and that `min_area` is missing.

What that costs: adding a new setting breaks older config files. I took that trade deliberately. The alternative is a typo being ignored while the program runs with a value nobody picked. That is exactly what the prototype did. Its `sport` fell back to `"soccer"` while the file said `"football"`, and `min_area` fell back to `500` while the file said `1000`. Nobody would have noticed, because nothing failed.

The three places I do fall back

All three are visible, and none of them invent data.

1. **No frame rate in the file, so look at every frame.** If the file does not say how fast it is, I cannot work out a stride. Falling back to "look at everything" costs more time but never throws data away. That is the safe direction. It logs a warning with the reason, so if a run is slower than you expected you can find out why.
2. **Cut short by up to 2% is fine.** Video files often report a frame count that is a little off, so finishing two frames early is normal. More than that and it is a broken file. Without a tolerance every run would fail. With too much tolerance, a genuinely damaged video would be reported as a success, which is the bug I was fixing.

3. **If the platform is down, the run still finishes.** Reports are retried with a backoff and then dropped. A circuit breaker stops it hammering a dead service, which would otherwise make a long run slower and slower. The run completes and the result says `reporting_degraded` with a count of what was lost. A failure to report never becomes the failure being reported.

Config I removed instead of copying

Dead settings are worse than no settings. They tell an operator that turning a dial does something when it does nothing. These were all in the prototype and none of them were ever read:

Old setting	What happened to it
<code>field_detector.type:</code> <code>"sam_mask_v1"</code>	Never read. Now it actually picks the detector, and an unknown name fails at startup.
<code>confidence_threshold: 0.5</code>	Stored in a variable and never used. Became <code>aggregation.min_confidence</code> , which is really enforced: anything below it is counted as a <code>low_confidence</code> rejection.
<code>debug_mode: True</code>	Never read. Became <code>logging.level</code> and <code>logging.format</code> , because a true/false cannot tell you the difference between quiet and diagnosable.
<code>crop_search</code> (the whole block)	Unused, and crop layout is not built. Left out rather than copied as decoration.
<code>target_fps: 30</code>	Never read, and every frame was decoded anyway. Now it drives the sampling.

One thing I did not "fix", on purpose

I ported the detector as-is, including a real flaw. This is the part I would most want you to read.

The fake background is solid green and the pitch is drawn as white lines on top of it. The detector looks for green. So on every frame that is not blank, **the green mask covers the whole frame**, and the largest outline is therefore the **edge of the frame, not the pitch**. I measured it:

blank frame	-> rejected
green close-up	-> DETECTED, area 919,601 (the frame)
noise frame	-> DETECTED, area 919,601 (the frame)
normal frame	-> DETECTED, area 919,601 (the frame)

All three non-blank cases give back the same polygon. That is why the prototype's "1776 boundaries found" is not 1776 pitch boundaries. It is 1776 frames that happened to contain green, and 98.7% is an artefact rather than a result. You can see it in the summary numbers too: across the whole run, the interquartile range of the metric is **exactly 0**.

I could have quietly made the detector better. I did not, because fixing it is a modelling job rather than a plumbing one, and doing it silently would hide the most important thing about this detector.

There is a test called `test_known_limitation_the_boundary_is_the_frame_not_the_pitch` that fails if this behaviour changes. So if someone improves the detector later, they have to do it on purpose and update the docs.

3. Performance

The measurements

	Old prototype	This
Frames decoded	1800	1800 (but stepped over with <code>grab()</code>)
Frames actually looked at	1800	300
Wall clock	47.6 s	2.5 s
Per frame	about 26 ms	-
Metric interquartile range	-	0 (see section 2)

About **19 times faster** on the same video, same detector, same machine.

Where that comes from, and what it costs

Sampling is the big win and the big risk. I read the real frame rate from the file (30fps) and work out a stride from the target (5fps), which gives 6. So five frames out of six are never looked at. For a summary that is the right call. For anything needing per-frame output it is wrong. That is why it is question 1 above.

`grab()` instead of `read()`. `read()` is really two steps: grab the frame, then decode it. The decode is the expensive half. Skipped frames get the cheap half only. Honest caveat: on most codecs you still cannot jump past a frame without touching it, and seeking by frame number is not reliable across all video formats, so I did not use seeking. This is a real saving, not a free lunch.

Doing repeated work once. The prototype rebuilt the same 1280x720 rectangle every single frame to calculate the overlap, and rebuilt its colour range arrays every frame too. Both are built once now, using the video's actual size. That also fixes a bug: on any video that was not 720p, the old code would have measured the wrong thing.

One pass instead of two. `max(contours, key=cv2.contourArea)` works out the area of every outline and then works out the winner's again. Now it is one pass that keeps the best one.

Cheap rejections first. The minimum size check now happens before any geometry work. The deliberately noisy frames in the test video used to become full-size polygons. Now they are thrown out for the cost of one area calculation.

Memory is bounded where it matters. The prototype added every polygon to a list for the whole run, so memory grew with the video length and a long enough run would eventually die. Here the polygons, which are the heavy things, are kept in a small ring buffer. The metric values are plain numbers and I keep all of them so the median is exact. At 8 bytes each that is about 1.4 MB for ten hours of video at 5fps, which is not worth throwing accuracy away to save.

Median and interquartile range, not an average. The video has fake detections built into it. An average lets those drag the headline number around. A median does not. The cost is that it is less sensitive to a genuine slow change during a run. If that turns out to be a real signal, it is worth revisiting.

Early exit is off by default. It is the only thing here that makes a *longer* video genuinely cheaper for the *same* work, so it is worth having. But it stops looking before the end, so it should be a choice, not the default.

What I did not get to

- **Dependencies are not pinned to exact versions.** `requirements.txt` says things like `opencv-python-headless>=4.10.0.84`, and on this machine that resolved to OpenCV **5.0.0**. I checked the generator and video reading work on it, but a submission that installs something different next month is not reproducible. I would pin exact versions with a lock file. I did not do it here because there is no CI to keep pins honest.
 - **No proper benchmark.** The 19x is one wall clock measurement, not a curve. A real benchmark would time decoding, detection, and adding up separately, and would use a much longer video to show how it scales.
 - **No profiler.** I found these wins by reading the prototype, not by profiling it. A profile would probably find different ones.
-

4. How I used AI

I used GitHub Copilot in agent mode throughout. I decided the design. The assistant wrote most of the code from that direction.

What it did

- Read the prototype and list its problems against the four parts of the brief. The table at the top of `docs/ARCHITECTURE.md` came from that.
- Wrote the config models, the detector seam, the frame scheduler, the aggregator, the reporter, the pipeline, and the CLI.
- Wrote most of the 176 tests, and the small scripts I used to check things by hand.
- Drafted this document and the README, from decisions I had already made and numbers we had already measured.

What I decided myself

- **The big one: port the detector as-is instead of fixing it, and write down the flaw loudly.** Left alone, the assistant would have quietly improved the detector. That would have hidden the most useful finding in the whole exercise.
- Fail hard on unknown config keys, and accept that adding a setting later breaks old configs.
- Not build crop layout, and delete its dead config block rather than keep it as decoration.
- Commit granularity, and pushing as I went. The brief grades git history, so the history had to be built while the work happened, not reconstructed at the end.
- Every trade-off in section 3.

Where the assistant was wrong

Its **claims** were much less reliable than its **code**, and the difference only showed up when I actually ran things.

- Six tests failed on the first run. All six were mistakes in the tests, not in the source code. The scheduler opens the video when you enter the `with` block, not when you create the object. The test frames were drawn at 720p coordinates on a 320x240 canvas, so the pitch outline got clipped into an open shape and split the green into separate pieces. The progress reporting test never fired because the test video only produces 30 frames and the default setting reports every 250. None of these were visible by reading the code.
- My truncation test gave exit 3 instead of 4. Truncating an MP4 file removes its `moov` atom, which makes the file impossible to open, which is exit 3. My expectation was wrong, not the code. The real "video cut short" path is now tested with a fake video reader instead.
- Some edits were applied incorrectly when I sent several at once to the same file, which produced Python that would not even parse. Found by running the parser, not by reading.
- One of my check scripts compared the wrong file paths and reported "DIFF" for two files that were actually byte for byte identical. The check was wrong, not the code.

The point of all that: **nothing in this document rests on the assistant saying so**. The 19x speedup, the zero interquartile range, the identical areas, and the exit codes were all measured after the fact. The assistant claimed the detector was broken just from reading the source. I only believed it once it was shown on real frames.

Not used: nothing generated by an LLM was left unchecked. I do use `ruff` for formatting and linting, but that is a tool, not generation.

5. Where I stopped

Done, tested, and containerised: config and its validation, the detector seam, sampling, aggregation, the failure rules, structured logging, platform reporting with degradation handling, the CLI, and the tests.

Left out on purpose

Not done	Why	Next step
Crop layout recommendation	Not required by parts 1 to 4, and the prototype's config for it was dead	Confirm it is wanted, then build it behind the same seam
Pinned dependency versions	No CI to keep pins honest	Add a lock file
A real benchmark	One measurement, not a curve	Time decode, detect and aggregate separately on a longer video
A real ML detector	The seam is there, the model is not	Add a <code>sam_mask_v1</code> config variant plus a registry entry. The test that checks config and registry agree will force it to be done properly
Auth on the reporting call	<code>mock_api</code> has none, and inventing one would be guessing	Ask what the platform expects
Processing only part of a video	Not asked for, and the prototype had no concept of it	A parameter on the frame scheduler. Cheap.

The first thing I would do with more time is get an answer to question 1. If per-frame analysis turns out to be required, the biggest performance decision in here is wrong, and everything else is small by comparison.

Architecture

Design notes for the pitch-boundary pipeline. This document is written **before** the code so the reasoning is reviewable on its own, and so each later commit can be read against the design it implements.

Reference point throughout: `synthetic_field_prototype.py`, imported unmodified in `2a85690`. Every decision below is a response to something specific in that script.

1. What the prototype gets wrong

Not a style critique — each row maps to a concrete risk if this ran unattended on long feeds. The right-hand column is the assignment part it violates.

#	Defect	Risk	Part
1	Config read via <code>.get(key, default)</code> . <code>sport</code> falls back to <code>"soccer"</code> while the config says <code>"football"</code> ; <code>min_area</code> falls back to <code>500</code> while the config says <code>1000</code> .	Silent divergence. The shipped file already contradicts itself.	1.2
2	<code>field_detector.type = "sam_mask_v1"</code> is never read. Detection is hardcoded to a green HSV threshold.	Dead config. Operators would believe they changed the detector and did not.	1.3
3	<code>_extract_mask</code> and <code>_derive_polygon_from_mask</code> are two halves of one responsibility.	No seam to swap detectors per sport/deployment.	1.3
4	<code>except Exception: pass</code> around contour extraction.	A <code>TypeError</code> or <code>MemoryError</code> is absorbed as "no boundary found" — a real bug becomes a data gap.	3.2
5	<code>Polygon([(0,0),(1280,0),(1280,720),(0,720)])</code> rebuilt every frame, dimensions hardcoded.	Redundant per-frame allocation; wrong for any feed that is not 1280x720.	2
6	<code>while True: cap.read()</code> decodes every frame. <code>target_fps: 30</code> never used. <code>time.sleep(0.005)</code> fake latency.	Cost scales with file length, not with work needed.	2
7	<code>detected_polygons</code> accumulates unbounded.	Memory scales with video length. OOM on a long feed.	2, 3.1
8	<code>cap.isOpened()</code> failure prints and returns.	Fatal error exits 0. Orchestrator sees success.	3.2
9	Mid-stream <code>ret=False</code> is indistinguishable from clean EOF.	A truncated/corrupt feed is reported as a completed run.	3.2
10	Invalid polygons silently dropped, uncounted, unreasoned.	No way to tell "no pitch visible" from "detector broke".	3.3
11	<code>print()</code> is the only observability. Nothing is reported over the network.	Unattended batch job with no monitorable state.	3.1, 4
12	<code>crop_search</code> config block is entirely unused (README confirms crop layout is unimplemented).	Config advertises capability that does not exist.	—

2. Target architecture

```
flowchart TB
    subgraph entry["Entry point (thin)"]
        CLI["cli.py<br/>argv -> load config -> run -> exit code"]
    end

    subgraph lib["Library (importable, testable)"]
        CFG["config.py<br/>pydantic settings, fail-fast loader"]
        MOD["models.py<br/>Detection, Progress, JobEvent, RunResult"]
        ERR["errors.py<br/>Fatal vs Recoverable taxonomy"]
        LOG["logging_setup.py<br/>structured JSON to stdout"]
        VID["video.py<br/>FrameScheduler"]
        PIPE["pipeline.py<br/>orchestration"]
        AGG["aggregation.py<br/>streaming, robust aggregation"]
        REP["reporting.py<br/>JobReporter"]
    end

    subgraph seam["The one seam that flexes"]
        BASE["detectors/base.py<br/>FieldDetector protocol + registry"]
        D1["green_threshold.py"]
        D2["sam_mask_v1.py"]
    end

    CLI --> CFG
    CLI --> LOG
    CFG --> PIPE
    PIPE --> VID
    VID --> BASE
    BASE -. "resolved by name" .-> D1
    BASE -. "resolved by name" .-> D2
    PIPE --> AGG
    PIPE --> REP
    AGG --> MOD
    REP --> MOD
    PIPE --> ERR
```

Dependency rule: `pipeline.py` depends only on abstractions it defines or imports from `detectors/base.py`. It never imports a concrete detector, never imports `requests`, and never touches `argparse`. Wiring happens once, in `cli.py`.

This is what makes the pipeline testable without a video file, a network, or OpenCV's detector behaviour.

3. Module responsibilities

Module	Owns	Explicitly does not own
<code>config.py</code>	Loading, parsing, validating, and reporting config errors	Any defaulting or coercion beyond declared validators
<code>models.py</code>	The data shapes that cross boundaries (detections, progress, events, results)	Business logic
<code>errors.py</code>	The exception hierarchy and its mapping to exit codes	Catching anything itself
<code>logging_setup.py</code>	Log record shape, run correlation id	Deciding what is fatal
<code>video.py</code>	Deciding <i>which</i> frames to decode and yielding them	Interpreting frame content
<code>pipeline.py</code>	Sequencing, counters, policy (fatal vs. continue), final result	Concrete detection, concrete transport
<code>aggregation.py</code>	Turning per-frame observations into a final robust summary	Reading video
<code>reporting.py</code>	Getting validated payloads to the platform, tolerating failure	Failing the run
<code>detectors/*</code>	Turning one frame into zero or one boundary	Anything about video, config files, or transport

4. The one seam

The assignment warns against a plugin system on every class, so exactly one interface is abstracted. Everything sport- and deployment-specific collapses behind it:

```
FieldDetector.detect(frame: np.ndarray) -> FieldDetection | None
```

`FieldDetection` carries the polygon, a confidence value, and the detector id that produced it — so a rejected or accepted detection can always be traced to its source.

Design choices:

- **One method, not three.** The prototype's two-step `_extract_mask` then `_derive_polygon_from_mask` is an implementation detail of *one* detector, not a contract. A future learned model returns a polygon directly and has no mask step.
- **None means "no boundary in this frame".** That is a normal, expected outcome on close-ups. It is distinct from raising.
- **Resolved by name from config.** `field_detector.type` is validated against the registry at config-load time, so an unknown detector is a startup failure rather than a runtime surprise. The dead `type` key from the prototype becomes load-bearing.

- **Per-detector config is a discriminated union** on `type`, so each implementation validates its own parameters (`min_area` for the threshold detector, model path and confidence for a future one) instead of a shared untyped bag.

Deliberately **not** abstracted: video reading, aggregation, reporting. Each has one real implementation today. Abstracting them would add indirection without buying a second case, which is the failure mode the assignment calls out.

5. Processing efficiency (Part 2)

The requirement is that cost scales with *how much of the video needs inspecting*, not with file length. Two separate costs, handled separately.

5.1 Don't decode what you don't analyze

A `FrameScheduler` reads the stream's real `CAP_PROP_FPS` and `CAP_PROP_FRAME_COUNT` and computes `stride = ceil(source_fps / target_fps)` — finally making `target_fps` meaningful instead of decorative.

Skipped frames advance with `cap.grab()`, not `cap.read()`. `read()` is `grab() + retrieve()`; `retrieve()` is the expensive decode-to-BGR step. Advancing without retrieving avoids that cost entirely. Measurable win at 50/60 fps sources, larger as source framerate grows.

Note the honest limitation: `grab()` still demuxes, and on inter-frame codecs you cannot truly skip decoding to a non-keyframe. Frame-accurate seeking via `CAP_PROP_POS_FRAMES` is not reliable across codecs. This is a real reduction in work, not a free lunch — recorded in `DECISIONS.md`.

5.2 Amortise expensive repeated work

- **Hoist loop constants.** The frame-boundary polygon moves out of the loop, and is derived from the *actual* frame dimensions read from the stream rather than the hardcoded `1280x720`.
- **Build detector constants once**, in `__init__` — HSV bound arrays, kernel sizes. The prototype rebuilds `np.array([35, 40, 40])` and friends on every single frame.
- **Cheap rejects before expensive work.** `max(contours, key=cv2.contourArea)` computes each contour's area, then the code builds a `Polygon` and does a Shapely intersection. Replaced with: one area pass, reject below `min_area` *before* polygon construction or spatial maths. Noise frames — which the generator injects deliberately — now cost almost nothing.
- **Streaming aggregation instead of accumulation.** The unbounded `detected_polygons` list is replaced by running statistics and a bounded ring buffer. Memory becomes $O(1)$ in feed length instead of $O(\text{frames})$.
- **Delete the fake latency.** `time.sleep(0.005)` modelled per-frame detector cost; the real detector provides it.

5.3 Optional convergence early-exit

Off by default, enabled by config. Once `min_valid_samples` are collected and the running metric's relative standard error drops below a threshold, processing stops. This is the only mechanism that makes a *longer*

feed genuinely cheaper for the *same* analysis, so it is worth having — but it is a correctness trade-off, not a pure win, so it never turns itself on.

6. Failure handling (Part 3)

6.1 Taxonomy

Every failure is classified once, in `errors.py`. Nothing is caught without being classified.

Class	Examples	Policy	Exit
<code>ConfigError</code>	unparsable YAML, <code>confidence_threshold: 5</code> , unknown detector type, unknown key	Fatal, before any I/O	2
<code>InputError</code>	file missing, <code>cap.isOpened()</code> false, zero frames	Fatal	3
<code>StreamError</code>	mid-stream <code>ret=False</code> before <code>frame_count</code> , repeated decode failure	Fatal — feed is truncated	4
<code>DetectionRejected</code>	empty mask, <3 points, degenerate or self-intersecting polygon, <code>area < min_area</code>	Recoverable , counted with a reason	—
<code>ReportingError</code>	connection refused, timeout, non-2xx	Never fatal , logged, retried, run continues	—
<code>UnexpectedError</code>	anything unclassified	Fatal , traceback logged, never absorbed	1

The prototype's `except Exception: pass` is replaced by a narrow `except (cv2.error, ValueError, GeometryError)` around the geometry step alone. Anything else propagates — a real bug must never be reclassified as a data gap.

6.2 Distinguishing truncation from clean EOF

The prototype treats `ret=False` as "video finished". Before concluding that, the scheduler compares frames actually read against `CAP_PROP_FRAME_COUNT`. If the stream ends materially early, or a decode error repeats, that is a `StreamError` — the run is truncated and must not be reported as a success.

6.3 Noisy frames and aggregation

Per frame: valid detection, or a rejection with a reason. Nothing else. Invalid frames are **counted and reason-tagged**, never silently dropped — that is what makes "no pitch visible" distinguishable from "detector broke".

Final output, from `aggregation.py`:

- counts: frames read, analyzed, skipped, valid, rejected (by reason)
- coverage ratio: `valid / analyzed`
- the spatial metric aggregated as **median and IQR, not mean** — the generator injects false-positive rectangles on purpose, and a mean would let them drag the headline number

- a consensus boundary polygon, plus the count of frames supporting it
- all-invalid is an explicit, non-crashing outcome with its own status and exit code

6.4 Observability

- One JSON object per line on stdout, carrying `run_id`, `level`, `event`, `frame_index`, `elapsed_ms`. Machine-readable because nobody is watching a console.
- Periodic progress reports to the platform every N frames or T seconds.
- Terminal events: `run.started`, `run.completed`, `run.failed` with a reason code and the aggregated result, so a failure is diagnosable after the fact from logs plus the platform.

7. Reporting (Part 4)

Payloads are built from pydantic models in `models.py` — the same family used for config — and serialised from those models. No dict assembled by hand anywhere near the wire.

The two failure directions are kept rigorously separate:

Situation	Pipeline	Reporting	Exit code
Everything fine	succeeds	succeeds	0
Platform unreachable	still succeeds	degrades, retries with backoff, breaker opens, drops counted	0, result flagged <code>reporting_degraded</code>
Pipeline fails	fails	best-effort <code>run.failed</code> event attempted before exiting	the <i>pipeline's</i> code (2/3/4/1)

Two rules enforce this:

1. `ReportingError` is caught **inside** `reporting.py` and returned as a status value. It never propagates into the pipeline's fatal path, so a broken endpoint cannot fail a healthy run.
2. A failure to report a failure never masks the original. The exit code is decided by the pipeline outcome first; reporting is attempted afterwards on a best-effort basis.

Timeouts are short and retries bounded, because stalling a batch job on a telemetry endpoint is the wrong default. Startup does a short bounded readiness probe and then proceeds regardless.

8. Configuration

`config/default.yaml` declares every field explicitly — no implicit defaults to fall back to. The loader:

1. reads the file; a parse error is a `ConfigError` naming file and line
2. validates against the pydantic model with `extra="forbid"`
3. raises `ConfigError` with the offending key path and the rejected value
4. exits 2 before any video is opened

`extra="forbid"` is a deliberate trade-off: a typo'd key fails loudly instead of being ignored. The cost is that adding a field is not backward compatible for older runners. Recorded in `DECISIONS.md`.

Environment overrides (`TRACKBOX_VIDEO_PATH` , `TRACKBOX_API_URL`) are validated through the same model — an override is not a bypass.

9. Exit codes

Code	Meaning
0	Completed successfully
1	Unexpected internal error
2	Configuration error
3	Input error (missing, unreadable, empty)
4	Stream error (truncated or corrupt feed)
5	Completed but no valid boundaries found

Deliberately distinct from one another: an orchestrator must be able to tell "bad config" from "bad video" from "ran fine, saw no pitch" without parsing text.

10. Testing

File	Covers
<code>test_config.py</code>	valid load; unknown key, bad type, out-of-range, malformed YAML, unknown detector — each asserts fail-fast, not fallback
<code>test_detectors.py</code>	green detector on synthetic green / noise / blank frames; registry resolves by name; unknown name fails
<code>test_scheduler.py</code>	stride arithmetic; skipped frames genuinely skipped; expected indices on a fixture
<code>test_aggregation.py</code>	median resists injected outliers; all-invalid yields the defined outcome; streaming stats match a batch computation
<code>test_reporting.py</code>	success; timeout degrades rather than fails; 5xx retried; breaker opens; payload validates
<code>test_pipeline.py</code>	end-to-end on a short generated video with a fake reporter; exit-code taxonomy; failures never silently absorbed

The pipeline tests use a fake reporter and a fake detector, proving the seam works for testing as well as for production flexibility.

11. Known limitations and open questions

Carried forward into `DECISIONS.md`, but flagged here so they are not discovered late:

1. **Frame sampling vs. exhaustiveness.** Sampling at `target_fps` means not every frame is classified. If a downstream consumer needs per-frame guarantees, this design is wrong and needs revisiting with the product team.
2. **Mid-stream resolution change.** Multi-camera cuts to a different resolution are not handled; the plan is to detect the change and fail fast rather than silently mix coordinate spaces.
3. **Crop layout scope.** The README describes the boundaries as feeding a downstream crop-layout step that is "not yet implemented"; Parts 1–4 never explicitly require it. Aggregation is implemented; crop suggestion is left as a documented stub behind the same seam unless told otherwise.
4. **Truncated-feed policy.** Abort (exit 4) is the chosen default, with partial results attached. An alternative is to process the prefix and mark it degraded — needs a product decision.